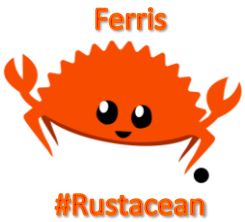
Introducción a Rust

El icono de Rust es un metal con una R



Rust = Oxido

Rusty = oxidado



La mascota “no oficial” es un cangrejo llamado **Ferris**

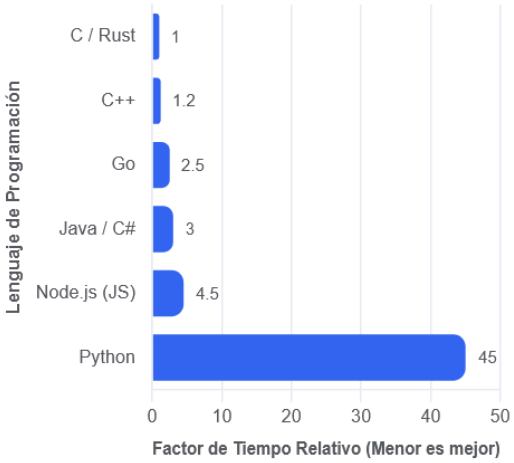
A los desarrolladores de Rust se les llama **Rustaceans**, en español “rustáceos”

Crustaceans = crustáceos

¿Qué es Rust?

Es un **lenguaje de programación** compilado, de propósito general, de código abierto enfocado en **seguridad de memoria**, **rendimiento** y **concurrencia**.

Eso quiere decir que **es muy rápido** y que su destino es construir sistemas operativos, sistemas embebidos en los que ejecutarse con un mínimo de recursos, hacer cálculos intensivos y llevarse bien con otros lenguajes para auxiliar en los casos en los que se necesita eficiencia y velocidad.



Es **multiplataforma** (se ejecuta en distintos sistemas operativos), **Multiparadigma** (permite diferentes enfoques de programación, POO, programación funcional, programación imperativa, …) y Multipropósito (se pueden crear diferentes tipos de aplicaciones, escritorio, web, controladores, …)

Multiplataforma



Multiparadigma



Multipropósito



Su origen fue la creación de un lenguaje para programas de sistemas 0 que dieran servicio a otros softwares. Esto hace que no esté enfocada en usuarios finales y generalmente no tiene interfaz gráfica.

Su primera versión es de 2010 y fue liberado en mayo de 2015 por **Mozilla**. En febrero de 2021, AWS, Huawei, Google, Microsoft y Mozilla anunciaron la formación de la Fundación Rust.

Rust es un lenguaje basado en C/C++ de tipado estático que ofrece una biblioteca estándar, que proporciona un rico conjunto de funciones, algoritmos y estructuras de datos para diversas tareas.

Características principales:

- ✓ **Alto rendimiento** →Es un lenguaje compilado a código máquina. Permite crear programas rápidos y eficientes, con un rendimiento comparable al de C y C++.

Genera ejecutables nativos optimizados para el hardware específico. (el compilador de Rust, 'rustc', utiliza LLVM (Low-Level Virtual Machine) como backend, lo que le permite aplicar una amplia gama de optimizaciones avanzadas.

- ✓ **Seguridad de memoria** → Destaca por su sistema de tipos, evita errores de memoria, como accesos a memoria liberada o referencias inválidas. Su compilador atrapa muchos errores comunes de programación en tiempo de compilación.

- ✓ **Gestión de memoria sin recolector de basura** →Utiliza un sistema de propiedad (*ownership*) y préstamos (*borrowing*) que permite gestionar la memoria sin errores comunes como null pointer (puntero que señala una variable que ha sido eliminada) o buffer overflows sin necesitar un recolector de basura tradicional. **Esto lo hace en tiempo de compilación** y evita errores antes de que el programa se ejecute.

- ✓ **Concurrencia segura** (hyperthreading) → Su sistema de tipos ayuda a detectar durante la compilación determinados errores relacionados con el acceso concurrente a los datos.

Rust evita condiciones de carrera en tiempo de compilación donde 2 o más hilos acceden y modifican la misma memoria al mismo tiempo sin la sincronización adecuada.

- ✓ **Abstracciones de alto nivel** → Ofrece estructuras de datos, funciones, genéricos, enumeraciones, patrones y características de programación funcional sin renunciar al control de bajo nivel.

- ✓ **Gestión de proyectos y bibliotecas** → Incluye **cargo**, su herramienta oficial para crear proyectos, compilar, ejecutar pruebas y gestionar dependencias.
- ✓ **Interoperabilidad** → Rust nos introduce al *Foreign Function Interface* (FFI) que consta de un mecanismo que permite a un lenguaje de programación llamar a funciones o utilizar estructuras de datos de otros lenguajes.
    - Permite utilizar bibliotecas existentes escritas en C, lo cual es invaluable dado el vasto ecosistema de bibliotecas C
    - Aunque de modo más complejo y con muchas restricciones, Rust puede interactuar con C++
    - Se pueden escribir extensiones de Python en Rust para mejorar el rendimiento.
    - Se puede compilar Rust a WebAssembly para usarlo en aplicaciones web.



Ejemplos de uso:

Rust se ha ganado el apoyo de grandes empresas como Mozilla, Microsoft, Dropbox, Amazon, Google, … que están utilizando Rust en proyectos importantes.

Mozilla Firefox utiliza Rust en sus componentes principales por seguridad y rendimiento. Mozilla fue realmente el creador del proyecto.

Utiliza Rust para mejorar el rendimiento y la seguridad en la gestión de grandes cantidades de datos.

Utiliza Rust en la creación de microservicios por su bajo consumo y alto rendimiento. **AWS Lambda** es un servicio de computación que permite ejecutar código en respuesta a eventos sin necesidad de gestionar ni aprovisionar servidores lo que resulta muy liviano y rápido.

Usa Rust para mejorar la seguridad en la memoria de sus sistemas y está probando para reemplazar el código C/C++ en Windows.

Esta plataforma de chat utiliza Rust en backend para mejorar la concurrencia y capacidad de respuesta cuando hay millones de usuarios conectados.



Bockchain (o cadena de bloques) es una DB digital descentralizado e inmutable que almacena información de forma bloques, ligados entre sí criptográficamente.

La idea con una analogía

Imagina un cuaderno de cuentas de un grupo de amigos:

- Cada vez que alguien paga o presta dinero, se anota una línea.(bloque con hash)
- **Todos** tienen una copia idéntica del cuaderno.
- Si alguien intenta falsificar su copia, las demás no coinciden y se descubre el engaño.

Eso es lo esencial: muchas copias iguales, vigiladas por todos.

Funciona como una base de datos compartida por miles de ordenadores (nodos) donde ninguna persona, banco o gobierno tiene el control total.

Para qué sirve

- **Criptomonedas** (Bitcoin, Ethereum): dinero digital sin banco central.
- **Contratos inteligentes**: programas que se ejecutan solos cuando se cumplen condiciones.
- **Trazabilidad**: seguir el origen de alimentos, medicamentos o productos.
- **Registros** de propiedad o certificados.

Alguna de estas Blockchain, como Solana, están escrtitas en Rust.

Desarrolla desde 2021 el sistema operativo **Fuchia SO** con Rust entre otros lenguajes. Se dice que puede ser el sustituto de Android o ChromeOS.

A partir de 2020, Linux empieza a usar Rust para desarrollar drivers de dispositivos por la seguridad de memoria y su integración con los sistemas. Rust sustituye en parte a C/C++

Redox OS es un sistema operativo completo escrito en Rust

Zed es un editor minimalista muy interesante.

Y muchas otras empresas como, por ejemplo, 1password, Huawei, Deno, …

Instalación de Rust

Desde la página oficial de Rust puedes ver como instalar dependiendo del sistema operativo que utilices, además, ofrece la posibilidad de probar código online en el Rust Playground



Rustup es un administrador de versiones y herramientas de Rust que facilita la instalación y administración de Rust y sus componentes relacionados.

En sistemas Unix (Linux, macOS):

1. Abre una terminal.
2. Ejecuta el siguiente comando para descargar e instalar:




```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Este comando descargará el script de instalación y lo ejecutará. Rustup se encargará del proceso de instalación y configuración.

3. Sigue las instrucciones en la terminal. Te pedirá que confirmes la instalación y configuración de Rust.
4. Una vez que se complete la instalación, cierra y vuelve a abrir la terminal o ejecuta el siguiente comando para que los cambios tengan efecto:

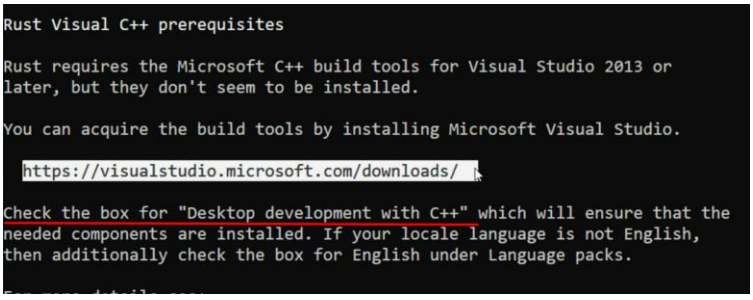
```
source $HOME/.cargo/env
```

En Windows:

1. Desde la página oficial de Rust, descarga el instalador y ejecuta.



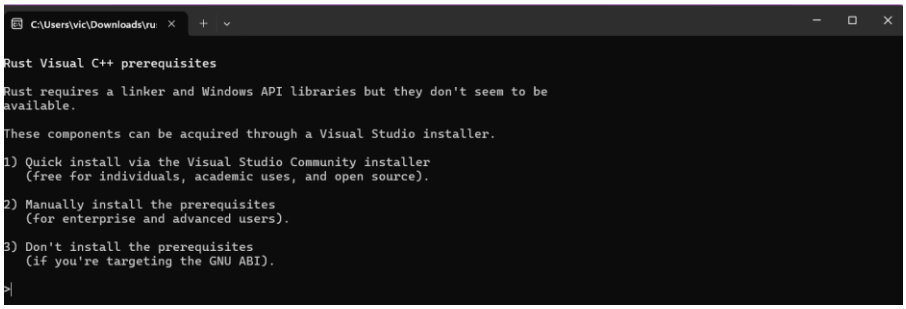
En la versión para Windows es obligatorio tener Microsoft C++ build toolls, si no lo está, el propio instalador indicará la URL para descargarla.



Hay que instalarlo y aunque ya lo esté hay que buscar “Visual Studio Installer” y modificarlo para marcar “Desarrollo para el escritorio C++”



Si Visual C++ está instalado, volvemos a ejecutar el instalador de Rust. Opción 1



Indica la ruta de instalación de los tres elementos necesarios:

⇒ **cargo** - el gestor de paquetes
⇒ **rustup** - para las actualizaciones de versión
⇒ **rustc** - el compilador

2. Instalamos con la opción 1 (por defecto).
3. Comprueba la instalación
   Reiniciamos la terminal y escribimos:

cargo - -versión →

```
C:\Users\vic>cargo --version
cargo 1.98.1 (797e8a9bc 2026-08-05)
```

rustup - -versión →

```
C:\Users\vic>rustup --version
rustup 1.29.1 (d95a37b6a 2026-08-13)
info: This is the version for the rustup toolchain manager, not the rustc compiler.
info: the currently active `rustc` version is `rustc 1.98.1 (48a229cea 2026-09-01)`
```

rustc - -versión →

```
C:\Users\vic>rustc --version
rustc 1.98.1 (48a229cea 2026-09-01)
```

** Si no lo reconoce hay que añadir la ruta en el Path.

4. **Actualizar Rust** (Si acabamos de instalar, no es necesario). Cada 6 semanas hay una versión nueva. `rustup update`

   ** rustup tiene otros comandos disponibles como, por ejemplo, uninstall o doc

`rustup self uninstall` `rustup doc` abre la documentación en un navegador

Proyecto de prueba desde consola

Identificadores → Las convenciones de nombres en Rust están en su documentación, en general se utiliza el estilo **snake_case** para los identificadores y reserva las variantes de **CamelCase** (Mayúsculas) para tipos de datos y estructuras (struct), Enums y Traits.

Rust es "Case Sensitive" → distingue mayúsculas y minúsculas.

cargo es una herramienta de línea de comandos, el sistema de compilación oficial y el gestor de paquetes o dependencias.

```
c:\appsRust>cargo new hello_word
    Creating binary (application) `hello_word` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
```

Esta orden genera una carpeta de aplicación independiente llamada *hello_word* con un archivo "**main.rs**" que se puede ejecutar desde la consola (en Linux → **./main**)

```
fn main() {
    println!("Hello, world!");
}
```

```
> cd hello_word
> rustc main.rs
> .\main

Hello, world
```

- Las funciones se definen con **fn**.
- Los bloques se delimitan con **llaves**, como en C y C++.
- La función **main** es el punto de entrada del programa.
- Rust tiene macros higiénicas (no sobrescriben variables externas), como por ejemplo **println!**
- Las cadenas están codificadas en **UTF-8** y pueden contener caracteres Unicode.

Ejecutar el Proyecto con cargo



cargo es una herramienta fundamental en Rust:

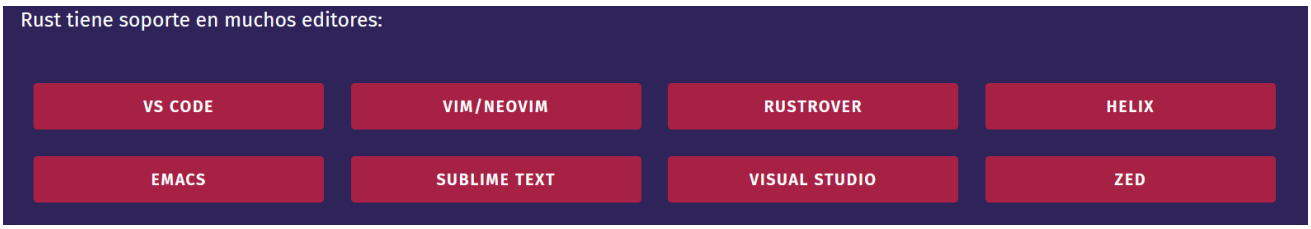
- ⇒ gestiona el proyecto
- ⇒ prueba el código
- ⇒ descarga las dependencias, si las hay
- ⇒ genera el binario
- ⇒ y lo ejecuta.

```
c:\appsRust>cd hello_word

c:\appsRust\hello_word>cargo run
   Compiling hello_word v0.1.0 (C:\appsRust\hello_word)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.51s
     Running `target\debug\hello_word.exe`
Hello, world!
```

IDE para Rust

Como menciona en su página Rust tiene soporte para diversos editores además de la consola.



Visual Studio Code

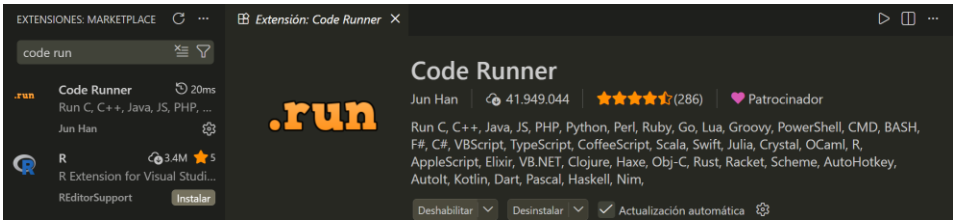
En este curso vamos a utilizar **VS Code** que es libre, muy intuitivo y ligero, además de multiplataforma.

Hay que instalarlo si no lo está ya. https://code.visualstudio.com/

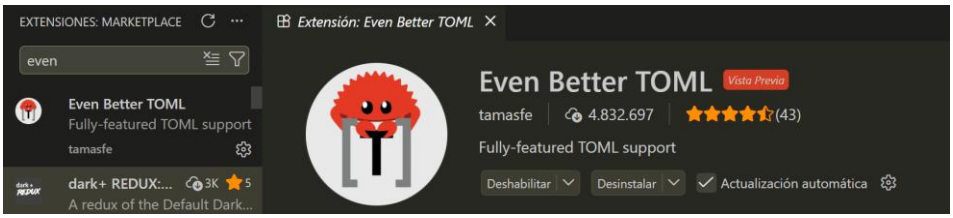
Extensiones:

rust-analyzer → soporte del lenguaje para VS Code, proporciona autocompletado inteligente, resaltado automático, detección de errores, advertencias en tiempo real, …





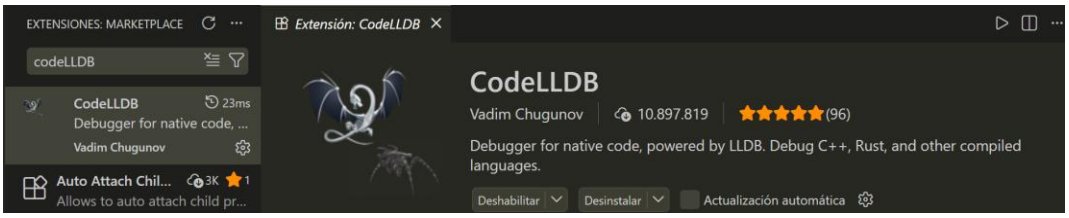
Even Better TOML para que el IDE formatee y valide lea los archivos con esta extensión (manifiesto)

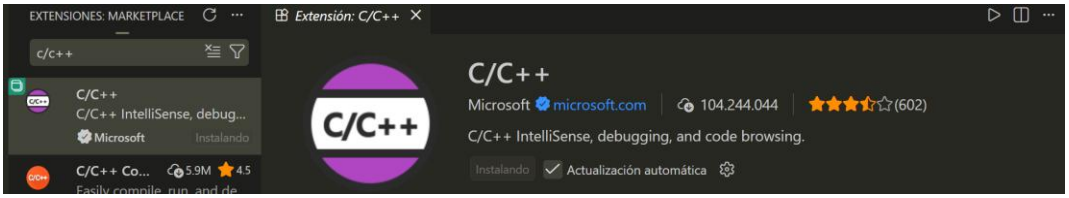


Gestor de dependencias

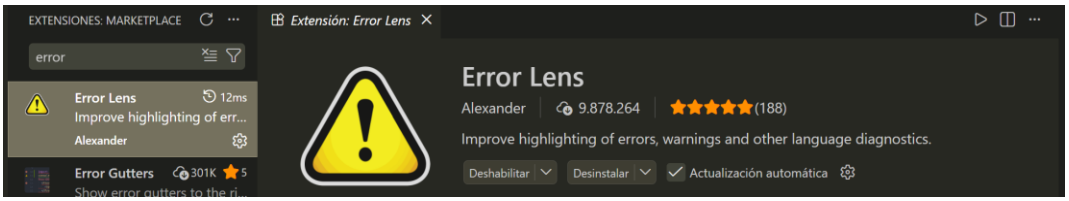


Depurador para código compilado

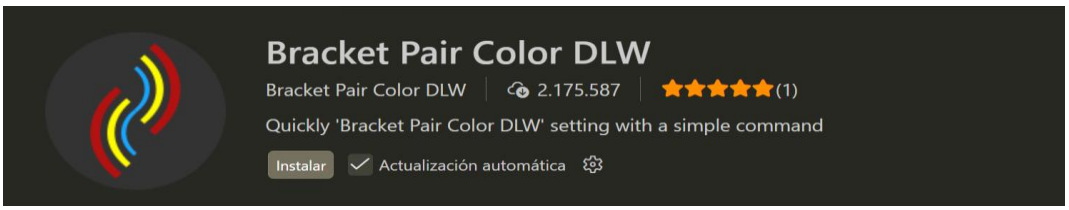




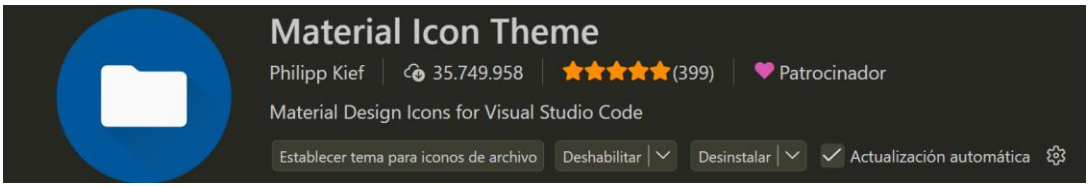
Ayuda en los errores



Colorear las llaves y los paréntesis



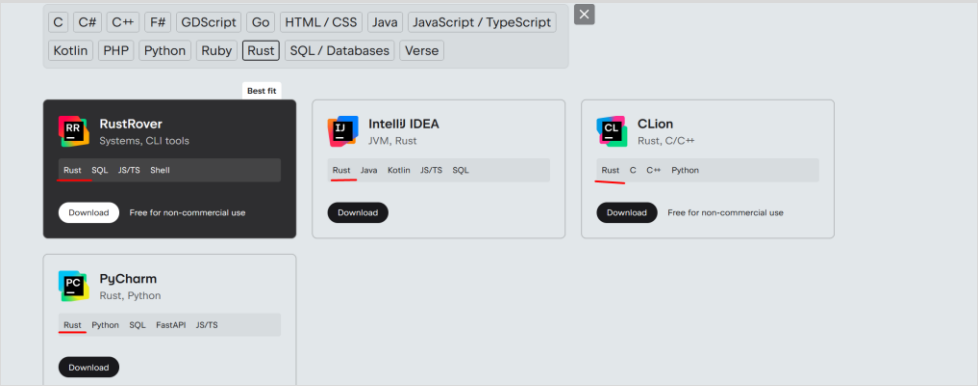
Iconos para los archivos



IDE alternativo

Rust en IntelliJ IDEA:

JetBrains tiene varios editores de código para Rust, el más específico es **RustRover** (no necesita añadidos para Rust pero es muy pesado, consume mucha RAM) y el más popular **Intellij IDEA** muy utilizado (sobre todo para java), de pago, pero se puede utilizar para uso no comercial



1. Asegúrate de tener **IntelliJ IDEA** instalado en tu sistema.

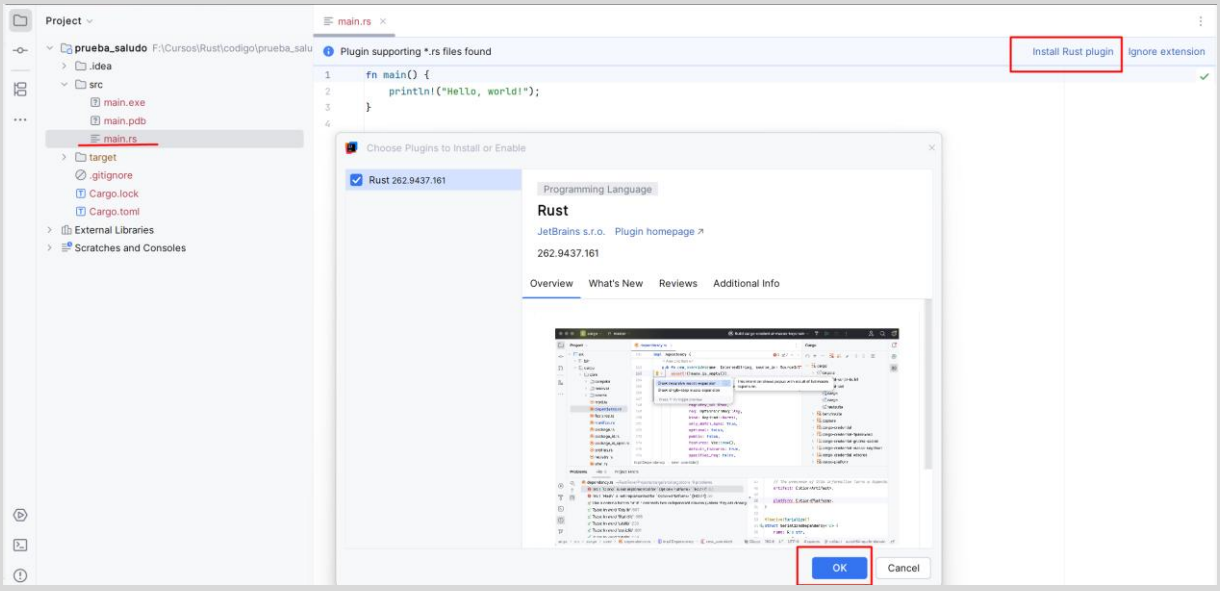
2. Abre IntelliJ IDEA y crea o abre un proyecto de Rust.

3. Ve al menú "File" (Archivo) -> "Settings" (Configuración) en Windows/Linux o "IntelliJ IDEA" -> "Preferences" en macOS.

4. En la ventana de configuración, habilita el plugin de "Rust".

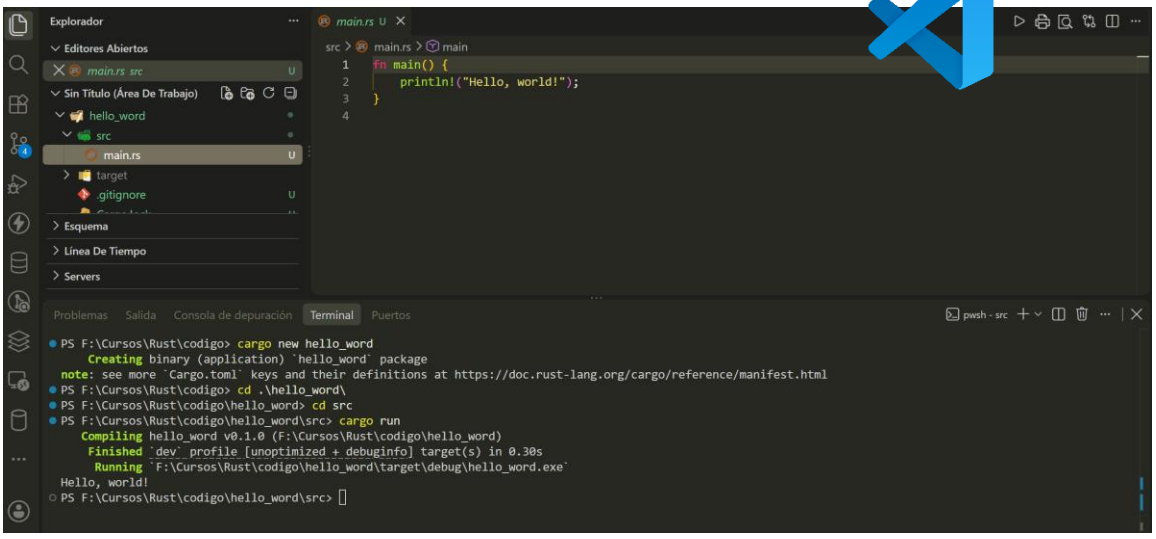
6. Después de instalar el plugin, reinicia IntelliJ IDEA si es necesario.

7. Si abres un proyecto con código te pedirá instalar el plugin



Práctica:

Abre el proyecto creado en consola y ejecutalo desde **VS Code



Modifica el archivo **main.rs** y escribe el siguiente código:

```rust
use ferris_says::say; // from the previous step
use std::io::{BufWriter, stdout};

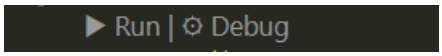
fn main() {
    let stdout = stdout();
    let message = String::from("Hola desde Rust!");
    let width = message.chars().count();

    let mut writer = BufWriter::new(stdout.lock());
    say(&message, width, &mut writer).unwrap();
}
```

```
 ___________________
< Hola desde Rust! >
 -------------------
        \
         \
            _~^~^~_
        \) /  o o  \ (/
          '_   -   _'
          / '-----' \
```

En el archivo Cargo.toml hay que añadir la dependencia

```toml
[dependencies]
ferris-says = "0.3.1"
```

Puedes ejecutar desde VS Code 

o desde el terminal

```
\hello_word> rustc src\main.rs
\hello_word> .\main
```